

Tecnatura Superior en Desarrollo de Software

Anexo 07c — El patrón Factory

Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat |
Ciclo Lectivo 2026

¿Por qué surge este anexo?

Contexto: en el Módulo 07 apareció esta línea en el `DependencyContainer` de `InterfaceAdapter`:

```
services.AddScoped<ICreateCustomerOutputPort>(  
    sp => sp.GetRequiredService<CreateCustomerPresenter>());
```

La expresión `sp => sp.GetRequiredService<...>()` es una **factory delegate**. Este anexo explica qué es una factory, por qué existe ese patrón, sus formas más comunes y cómo se aplica específicamente en el contenedor de DI.

¿Qué es el patrón Factory?

Factory

Patrón creacional — delega
la construcción de objetos

Patrón de diseño: Creacional

Un **Factory** (fábrica) es cualquier mecanismo que centraliza la *creación* de objetos. En lugar de que el código que necesita un objeto lo instancie directamente con `new`, delega esa responsabilidad a otro lugar: un método, una clase o una función anónima (delegate).

El resultado es que el código que *usa* el objeto no sabe **cómo** se construye — solo sabe qué tipo pide. Esto permite cambiar la construcción sin modificar el código cliente, y habilita decisiones en tiempo de ejecución: qué implementación concreta crear, con qué parámetros, reutilizando qué instancia ya existente, etc.

Formas comunes:

- **Factory Method** — método estático o virtual que crea el objeto
- **Factory Class** — clase dedicada a la construcción
- **Factory Delegate** — función anónima que devuelve el objeto
- **Abstract Factory** — familia de factories para familias de objetos

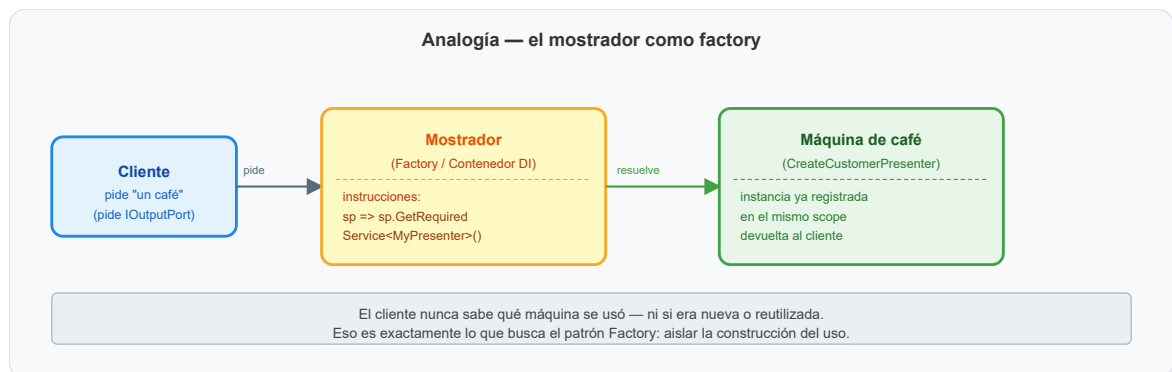
De las cuatro formas, la que aparece en el contexto del contenedor DI es la **Factory Delegate**: una función anónima (lambda) que el contenedor invoca cada vez que necesita resolver el tipo registrado. Es la forma más liviana y la más frecuente en proyectos .NET modernos.

Analogía del mundo real

Pensá en una cafetería de aeropuerto con autoservicio. Hay una máquina que hace café, otra que hace té y otra que hace jugo. Cuando querés una bebida, no fabricás la máquina — simplemente le decís al mostrador qué querés y él *sabe a qué máquina ir*.

El mostrador es la factory: tiene las instrucciones para obtener cada bebida. Vos — el cliente — solo pedís el tipo. Si mañana la máquina de café se reemplaza por una mejor, el mostrador actualiza sus instrucciones; vos seguís pidiendo "un café" exactamente igual.

En el código: el *cliente* es el constructor de una clase que pide `IOutputPort`. El *mostrador* es el contenedor DI. Las *instrucciones del mostrador* son la factory delegate: `sp => sp.GetRequiredService<MyPresenter>()`.



Las tres formas más comunes

1. Factory Method — método estático que construye

La forma más simple: un método estático en la propia clase que controla cómo se crea la instancia. Sirve para ocultar el constructor (haciéndolo privado) y aplicar validaciones o lógica antes de devolver el objeto. Es el mismo patrón que ya usaste en `OperationResult<T>` con `Ok()` y `Fail()`.

```

1  // ejemplo dominio neutro: creación de un punto en un plano cartesiano
2  public sealed class Point
3  {
4      // constructor privado — nadie puede hacer new Point() desde afuera
5      private Point(double x, double y) { X = x; Y = y; }
6      public double X { get; }
7      public double Y { get; }
8
9      // Factory Method cartesiano — coordenadas directas
10     public static Point FromCartesian(double x, double y)
11         => new(x, y);
12 
```

```
13 // Factory Method polar – convierte antes de construir
14 public static Point FromPolar(double radius, double angle)
15     => new(Math.Cos(angle) * radius,
16           Math.Sin(angle) * radius);
17 }
18
19 // uso – el cliente no sabe si usó coordenadas cartesianas o polares internamente
20 var p1 = Point.FromCartesian(3, 4);
21 var p2 = Point.FromPolar(5, 0.927);
```

Esto ya lo conocés: en el módulo anterior creaste `OperationResult<T>`. `Ok(value)` y `OperationResult<T>.Fail(errors)`. Ambos son Factory Methods: el constructor es privado y solo ellos pueden crear instancias.

2. Factory Class — clase dedicada a construir

Cuando la lógica de construcción es compleja o necesita dependencias propias, se extrae a una clase separada. Esta clase tiene un único propósito: saber cómo construir un tipo determinado. Implementa una interfaz `IFactory<T>` con un método `Create()`.

```
1 // ejemplo: generador de reportes – la construcción necesita una conexión
2 public interface IReportFactory
3 {
4     Report Create(ReportType type);
5 }
6
7 public class ReportFactory : IReportFactory
8 {
9     private readonly IDataSource _source;
10
11     public ReportFactory(IDataSource source) => _source = source;
12
13     public Report Create(ReportType type)
14         => type switch
15         {
16             ReportType.Pdf => new PdfReport(_source),
17             ReportType.Excel => new ExcelReport(_source),
18             _ => throw new ArgumentException("tipo no soportado")
19         };
20 }
```

3. Factory Delegate — la lambda del contenedor DI

Es la forma que apareció en el Módulo 07. En lugar de una clase separada, la "fábrica" es directamente una función anónima (lambda / delegate) que se le pasa al contenedor DI en el momento del registro. El contenedor la invoca cada vez que necesita resolver el tipo.

La firma de la lambda siempre es `IServiceProvider sp => <expresión que devuelve el objeto>`. El parámetro `sp` es el propio contenedor, lo que permite pedir otros servicios ya registrados para construir el objeto. Esto es lo que hace que la factory delegate sea tan poderosa en DI: puede *reutilizar* una instancia que ya existe en el scope en lugar de crear una nueva.

```
1 // registro normal – el contenedor crea la instancia por sí solo
2 services.AddScoped<OrderPresenter>();
3
4 // registro con factory delegate – le decimos AL CONTENEDOR cómo construirlo
5 services.AddScoped<IPlaceOrderOutputPort>()
6     sp => sp.GetRequiredService<OrderPresenter>();
7
8 // sp es el IServiceProvider – tiene acceso a todos los servicios del scope
9 // GetRequiredService devuelve la instancia ya registrada de OrderPresenter
10 // al ser Scoped, es la MISMA instancia dentro del mismo request
```

¿Qué está pasando en esas dos líneas?

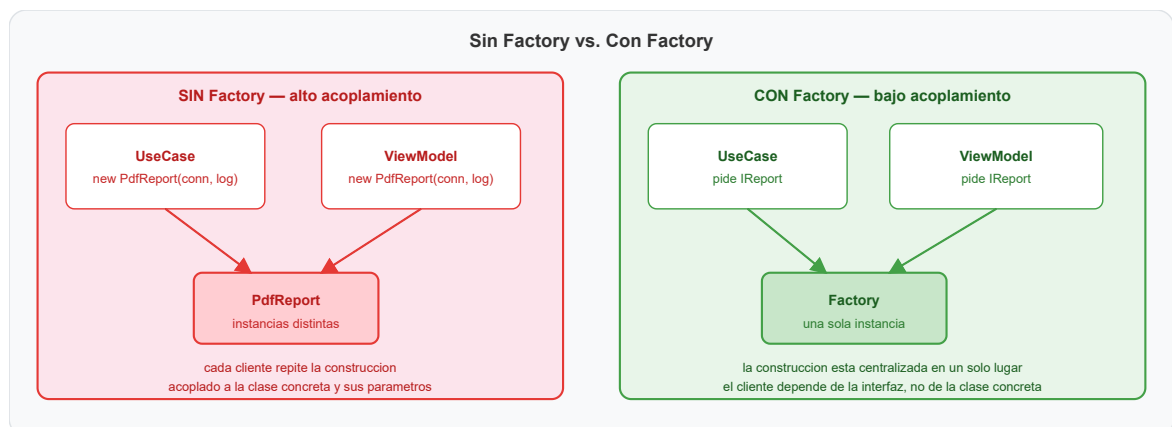
Línea 2 — `AddScoped<OrderPresenter>()`: el contenedor registra `OrderPresenter` con ciclo de vida `Scoped`. Cuando alguien pide un `OrderPresenter`, el contenedor crea una instancia nueva y la reutiliza durante todo el request.

Línea 5-6 — `AddScoped<IPlaceOrderOutputPort>(sp => ...)`: el contenedor registra `IPlaceOrderOutputPort` con una factory delegate. Cuando alguien pide un `IPlaceOrderOutputPort`, el contenedor ejecuta la lambda — que a su vez pide el `OrderPresenter` que ya está en el scope.

Resultado: tanto el UseCase (que inyecta `IPlaceOrderOutputPort`) como el ViewModel (que inyecta `OrderPresenter`) obtienen **la misma instancia**.

¿Por qué se usa?

El patrón Factory resuelve un problema fundamental en el diseño de software: **el código que usa un objeto no debería saber cómo construirlo**. Cuando el código cliente instancia objetos directamente con `new`, queda acoplado a la clase concreta, a sus parámetros y a cualquier dependencia que esa clase necesite.



En el contexto del contenedor DI, el factory delegate tiene un beneficio adicional y muy específico: permite **reutilizar una instancia ya registrada** en lugar de crear una nueva. Esto resuelve el problema de la instancia compartida entre el UseCase y el ViewModel que viste en el Módulo 07.

¿Cómo encaja en la arquitectura?

Los tres sabores de Factory aparecen en capas distintas:

Domain — Factory Method con constructor privado.

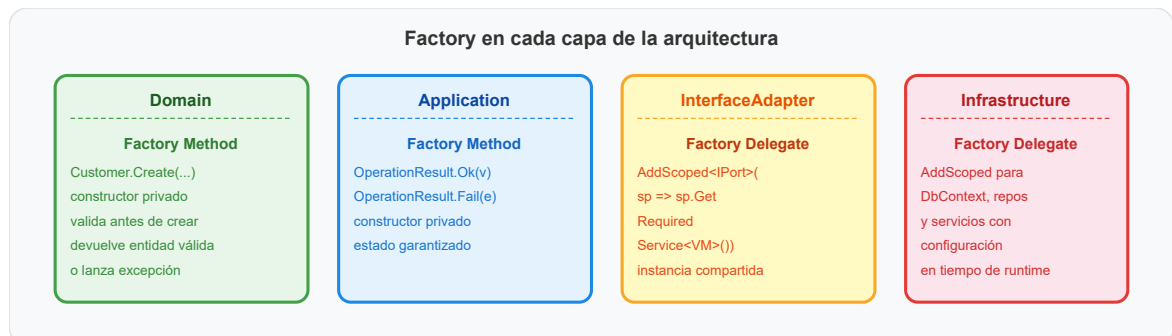
Las entidades y los Value Objects usan Factory Methods para garantizar que nunca existan en estado inválido. Ejemplo: `Customer.Create(...)`, `OperationResult.Ok(...)`.

Application — Factory Method en DTOs o resultados.

`OperationResult<T>.Ok()` y `Fail()` son Factory Methods que impiden construir resultados indeterminados.

InterfaceAdapter / Infrastructure — Factory Delegate en DI.

El registro con lambda en `DependencyContainer.cs` permite compartir instancias entre el UseCase y el ViewModel dentro del mismo scope.



El factory delegate en detalle

¿Qué es un delegate en C#?

Un **delegate** en C# es un tipo que representa una referencia a un método — es decir, una variable que apunta a una función. Los delegates permiten pasar funciones como parámetros, almacenarlas en campos, o asignarlas como valores.

La sintaxis de lambda `=>` es la forma abreviada de escribir un delegate anónimo (sin nombre). En el registro DI, la firma del delegate que acepta `AddScoped` es:

```

1 // el tipo del delegate que AddScoped acepta como parámetro
2 Func<IServiceProvider, TService>
3
4 // escrito de forma explícita – sin lambda
5 Func<IServiceProvider, IPlaceOrderOutputPort> factory
6     = (sp) => sp.GetRequiredService<OrderPresenter>();
7
8 services.AddScoped<IPlaceOrderOutputPort>(factory);
9
10 // la forma abreviada con lambda – equivalente exacto de lo anterior
11 services.AddScoped<IPlaceOrderOutputPort>(
12     sp => sp.GetRequiredService<OrderPresenter>());
  
```

¿Cuándo se ejecuta la lambda?

La lambda **no se ejecuta en el momento del registro**. Se registra como instrucción y se ejecuta *cada vez* que el contenedor necesita resolver `IPlaceOrderOutputPort`. Para un registro `Scoped`, eso ocurre una vez por request.

Cronología dentro de un request HTTP:

1. Llega el request — el contenedor crea un nuevo scope.
2. El ViewModel es instanciado — pide `OrderPresenter`. El contenedor crea la instancia y la almacena en el scope.
3. El UseCase es instanciado — pide `IPlaceOrderOutputPort`. El contenedor ejecuta la lambda: llama a `sp.GetRequiredService<OrderPresenter>()`, encuentra la instancia ya creada en el scope y la devuelve.
4. UseCase y ViewModel tienen la **misma instancia**.
5. Termina el request — el scope se destruye y la instancia se libera.

Errores comunes

ERROR COMUN

Usar `new` dentro de la lambda:

```
1 // MAL — crea una instancia nueva, rompe el scope
2 services.AddScoped<IPlaceOrderOutputPort>()
3     _ => new OrderPresenter();
```

El ViewModel recibirá un `OrderPresenter` vacío — el UseCase escribió en otra instancia creada aquí.

BUENA PRACTICA

Recuperar la instancia del scope con `GetRequiredService`:

```
1 // BIEN — reutiliza la instancia del scope
2 services.AddScoped<IPlaceOrderOutputPort>()
3     sp => sp.GetRequiredService<OrderPresenter>();
```

ERROR COMUN

Registrar el Presenter antes de registrar el OutputPort:

```
1 // MAL – el orden no importa en este caso, pero
2 // si el Presenter no está registrado, GetRequiredService lanza
3 services.AddScoped<IPlaceOrderOutputPort>{
4     sp => sp.GetRequiredService<OrderPresenter>();
5 // falta AddScoped<OrderPresenter>() – lanzará InvalidOperationException
```

BUENA PRACTICA

Registrar siempre el tipo concreto antes de la factory:

```
1 // BIEN – el tipo concreto va primero
2 services.AddScoped<OrderPresenter>();
3 services.AddScoped<IPlaceOrderOutputPort>{
4     sp => sp.GetRequiredService<OrderPresenter>();
```

ERROR COMUN

Confundir Factory Method con constructor:

```
1 // MAL – el constructor público expone la construcción sin control
2 public OperationResult(bool ok, T? v, IReadOnlyList<string> e) { ... }
3
4 // alguien puede crear un estado inválido: IsSuccess=true sin valor
5 new OperationResult<int>(true, null, []);
```

BUENA PRACTICA

Constructor privado + Factory Methods que garantizan estados válidos:

```
1 // BIEN – solo Ok() o Fail() crean el objeto
2 private OperationResult(...) { ... }
3 public static OperationResult<T> Ok(T v) => new(true, v, []);
4 public static OperationResult<T> Fail(string e) => new(false, default, [e]);
```

Qué NO hacer

QUE NO HACER

Poner lógica de negocio dentro del factory delegate:

```
1 // NO – la factory no es el lugar para tomar decisiones de negocio
2 services.AddScoped<IPlaceOrderOutputPort>(sp =>
3 {
4     var config = sp.GetRequiredService<IConfiguration>();
5     if (config["mode"] == "advanced")
6         return new AdvancedPresenter();
7     return new SimplePresenter();
8 });
```

Si necesitas selección dinámica de implementación, usá una **Factory Class** registrada en DI y con su propia interfaz. El delegate es solo para composición liviana.

BUENA PRACTICA

El delegate solo redirige — sin lógica propia:

```
1 // BIEN – delega al tipo concreto ya registrado
2 services.AddScoped<OrderPresenter>();
3 services.AddScoped<IPlaceOrderOutputPort>(
4     sp => sp.GetRequiredService<OrderPresenter>());
```

Resumen

Factory Method — método estático con constructor privado. Controla la construcción de un objeto garantizando invariantes. Aparece en Domain (entidades, Value Objects) y Application (OperationResult).

Factory Class — clase dedicada con interfaz propia. Centraliza construcción compleja con dependencias. Aparece cuando la lógica de creación es suficientemente grande para merecer su propia clase.



Factory Delegate — lambda pasada al contenedor DI. Permite reutilizar una instancia ya registrada en el scope. Aparece en InterfaceAdapter y Infrastructure para resolver el problema de la instancia compartida entre UseCase y ViewModel.

Regla práctica: si el factory delegate hace más que una sola línea de redirección, el código pertenece a una Factory Class.

Anexo 07c — El patrón Factory | Programación II | IES N.º 9007 Dr. Salvador Calafat
| Ciclo Lectivo 2026